Universidad Rafael Landívar
Campus San Alberto Hurtado, S.J. de Quetzaltenango
Curso: Programación Web
Docente: Ing. Manuel Rojas
Título: Eventia: de una api insegura a un diseño seguro

Nombre: _____ Carné: _____

CASO: EVENTIA Y SU PRIMERA AUDITORÍA DE SEGURIDAD

Eventia es una plataforma web que permite a los usuarios descubrir eventos, consultar disponibilidad e inscribirse en diferentes actividades. Los organizadores pueden publicar eventos, administrar sus actividades y consultar a las personas inscritas.

Durante los últimos meses, el equipo de desarrollo se concentró en implementar las funcionalidades principales. La plataforma funciona, pero las pruebas se realizaron principalmente desde la interfaz de usuario y no se revisó adecuadamente la seguridad de las APIs. Antes de publicar una nueva versión, la empresa solicita a su equipo realizar una auditoría y proponer las correcciones necesarias.

1) Auditoría de las apis existentes

Analicen las siguientes siete APIs. Cada una presenta uno o varios problemas relacionados con la seguridad.

API 1. Consulta de inscripciones
Endpoint: GET /api/inscripciones/125
Respuesta actual:

- id: 125
- eventoId: 20
- usuarioId: 35
- nombre: Carlos
- correo: carlos@email.com

El backend únicamente comprueba que exista la inscripción. No verifica si pertenece al usuario que realiza la solicitud.

API 2. Registro de usuarios
Endpoint: POST /api/usuarios
Datos recibidos:

- nombre: Ana
- correo: ana@email.com
- password: 123456
- rol: administrador

El backend guarda directamente todos los campos recibidos, incluyendo el rol indicado por el cliente.

API 3. Creación de eventos
Endpoint: POST /api/eventos
Datos recibidos:

- titulo: Concierto universitario
- cupo: -50
- precio: -100

El backend no valida los valores y permite registrar el evento.
Además, cualquier usuario autenticado puede crear eventos, aunque no sea organizador.

API 4. Modificación de eventos
Endpoint: PUT /api/eventos/25
La aplicación verifica que el usuario tenga el rol de organizador, pero no comprueba si el evento le pertenece.
Un organizador puede modificar eventos publicados por otros organizadores.

API 5. Inscripciones
Endpoint: POST /api/eventos/25/inscripciones
Datos recibidos:

- cantidad: 500

La aplicación permite registrar solicitudes sin establecer un máximo de inscripciones por usuario.
Tampoco comprueba de manera segura los cupos disponibles cuando varias personas intentan inscribirse simultáneamente.

API 6. Consulta de usuarios
Endpoint: GET /api/usuarios/35
Respuesta actual:

- id: 35
- nombre: Carlos
- correo: carlos@email.com
- telefono: 55555555
- passwordHash: valor almacenado
- rol: usuario

El frontend solamente utiliza el nombre y el correo, pero el backend devuelve todos los campos almacenados.

API 7. Inicio de sesión
Endpoint: POST /api/auth/login
El endpoint permite intentos ilimitados de inicio de sesión.
Además, cuando el correo no existe, responde: "El correo no está registrado".
Cuando la contraseña es incorrecta, responde: "La contraseña es incorrecta".
Esto permite distinguir si una cuenta existe.

Identificación de vulnerabilidades; Analicen las siete APIs.
Por cada una deberán identificar:

1. ¿Qué está mal en su diseño?
2. ¿Qué consecuencias podría tener?
3. ¿Con qué categoría del OWASP API Security Top 10 se relaciona?
4. ¿Qué control de seguridad debería implementarse?

Utilicen la siguiente estructura:

API	Problema identificado	Riesgo OWASP API	Solución propuesta
API 1			
API 2			
API 3			
API 4			
API 5			
API 6			
API 7			

Pueden consultar la referencia oficial: OWASP Api Security

Importante: Algunas APIs presentan más de un riesgo. Deben identificar el principal y pueden agregar otros si justifican su relación.

2) Rediseño

Una vez identificados los problemas, deberán diseñar nuevamente las siete operaciones.
Para cada endpoint definan:

1. Método HTTP y ruta.
2. Función que realiza.
3. Quién puede utilizarlo.
4. Datos que recibe.
5. Validaciones necesarias.
6. Controles de autorización.
7. Respuesta esperada.
8. Posibles códigos HTTP de error.

Ejemplo:
Endpoint: POST /api/eventos
Acceso: Organizador autenticado.
Datos recibidos:

- titulo: Concierto universitario
- cupo: 150
- precio: 75

Controles de seguridad:

- Validar autenticación.
- Verificar rol autorizado.
- Validar que el título no esté vacío.

- Validar que el cupo sea un entero positivo.
- Validar que el precio no sea negativo.
- Rechazar propiedades no permitidas.

Respuestas esperadas:

- 201 Created: evento creado.
- 400 Bad Request: datos inválidos.
- 401 Unauthorized: autenticación ausente o inválida.
- 403 Forbidden: usuario sin permiso.

3) Definición de permisos
   Construyan una matriz de autorización considerando los siguientes roles:
   - Visitante.
   - Usuario registrado.
   - Organizador.
   - Administrador.

Determinen qué roles pueden realizar las siguientes operaciones:

1. Consultar eventos públicos.
2. Crear eventos.
3. Modificar eventos propios.
4. Modificar eventos ajenos.
5. Inscribirse en eventos.
6. Consultar inscripciones propias.
7. Consultar inscripciones de un evento como organizador.
8. Administrar usuarios.

Para cada operación indiquen si el acceso está permitido o denegado. Cuando corresponda, expliquen las condiciones adicionales necesarias.

Importante: Deben diferenciar entre tener un rol y estar autorizado para operar sobre un recurso específico.

4) Documentación con openapi

Documenten al menos tres de los endpoints corregidos en la fase anterior.
Cada operación debe incluir:

1. Método HTTP y ruta.
2. Descripción de la operación.
3. Parámetros o cuerpo de solicitud.
4. Esquema de respuesta.
5. Códigos HTTP esperados.
6. Requisito de autenticación cuando corresponda.

Condición**:** La documentación debe coincidir con los permisos y validaciones definidos en la Fase 2.

5) Utilizando el proyecto Eventia desarrollado en las clases anteriores, implementen al menos dos controles de seguridad de los propuestos durante la auditoría.

Pueden trabajar con las funcionalidades que ya tengan construidas.
No es necesario implementar las siete APIs completas durante esta actividad.
Por cada corrección deberán presentar:

1. Comportamiento anterior.
2. Problema de seguridad identificado.
3. Modificación realizada.
4. Prueba con datos válidos.
5. Prueba con datos inválidos o acceso no autorizado.
6. Resultado obtenido.

Las pruebas pueden realizarse utilizando Postman, Thunder Client o una herramienta equivalente.

Ejemplo: Intentar registrar un evento con cupo negativo y comprobar que el backend rechaza la solicitud.

Respondan:

1. ¿Cuál de las siete APIs representa el mayor riesgo para Eventia? Justifiquen.
2. ¿Por qué utilizar JWT no garantiza que una API esté correctamente protegida?
3. ¿Qué diferencia existe entre validar los datos recibidos y autorizar una operación?
4. ¿Por qué documentar la autenticación en OpenAPI no significa que la seguridad ya esté implementada?
5. ¿Qué controles deberían mantenerse cuando Eventia evolucione y comience a utilizar servicios independientes?

Entregables:

1. Auditoría de las siete APIs, incluyendo problemas, consecuencias y riesgos OWASP.
2. Propuesta de rediseño de las siete operaciones.
3. Matriz de roles y permisos.
4. Documentación OpenAPI de tres endpoints.
5. Evidencias de dos controles implementados y probados.
6. Enlace al repositorio actualizado de Eventia.
7. Respuestas a las preguntas finales.